GEMINI ARGON — OPERATION SPARTAN | COMPLETE SYSTEMS & PERFORMANCE FORENSIC INVESTIGATION

God of War III Remastered (PS4 CUSA01623) — Native ARM64 Apple Silicon

Target: /Users/ihabfawaqa/Downloads/gow3-arm64-runtime-int @ HEAD 83e4d4b | Host: Apple M2 MacBook Air (4 P-cores + 4 E-cores, Passive Thermal Envelope) | Stack: Static x86-64->ARM64 Recomp + GoW3 shadPS4 Vulkan + Mesa KosmicKrisp -> Metal

Part A — Executive Verdict: The Five Strongest Findings at HEAD 83e4d4b

Executive Verdict: At current HEAD (`83e4d4b`), *God of War III Remastered* plateaus at **37–41.3 FPS** in the ~710-draw Mount Olympus scene (`bench_diag.csv`: 24.19 ms mean; `bench_enc.csv`: 24.53 ms mean) and **29.3–34.3 FPS** in ~1,500-draw combat (`bench_sub.csv`: 29.14 ms mean; `bench_unr.csv`: 34.17 ms mean). Cross-correlating the source code at `83e4d4b` with all 8 per-flip CSV traces in `docs/handoff/bench/bench_*.csv` and the new driver diagnostics (`METAL_ENCODERS_20261026.md` and `SUBMIT_COALESCING_PROPOSAL_20261026.md`) reveals the **five strongest, source-verified root causes** of the performance ceiling, stage-transition hitches, and audio/cutscene anomalies:

- **1. VBlank-Gated Display Buffer Lock (`port->buffer_labels`) Stalls `GpuCommandProcessor` in `WaitRegMem` for 6.50–11.76 ms Every Frame (`driver.cpp:643-648, 747, 811` & `liverpool.cpp:1299-1340`):** Across all 8 benchmark CSVs at `83e4d4b`, **96–99% of gameplay frames are strictly quantized to 16.67 ms (60 FPS), 33.33 ms (30 FPS), or 50.00 ms (20 FPS)**, with **exactly 1.0 `regmem_wait` per flip consuming 6.50 to 11.76 ms/frame** and `submit_lock_ns` **blocking** `guest-main` **for 14.01 to 22.06 ms/frame**! Why? When `sceGnmSubmitAndFlipCommandBuffers` runs, `PatchFlipRequest` (`gnmdriver.cpp:2140-2146`) injects a PM4 `WriteData` setting `port->buffer_labels[buf_idx] = 1`. When the guest reuses that display buffer, the PM4 stream issues `WaitRegMem` (`liverpool.cpp:1299`) polling `port->buffer_labels[buf_idx] == 0`. Even though `SubmitFlipInternal` (`driver.cpp:685-706`) has **already composited/copied the guest display buffer into a private `Vulkan::Frame*` image inside `PrepareFrame()`, `port->buffer_labels[prev_index]` **is** not reset to `0` until `VideoOutDriver::Flip()` runs (`driver.cpp:644-648`)! And because `immediate_flips = frame_limit != 0` (`driver.cpp:747`) **is** `false`** when `frame_limit == 0`, `PresentThread` ignores `request_cv.notify_one()` and sleeps in `timer.End()` (`line 811`) until the next 16.67 ms VBlank boundary—freezing `GpuCommandProcessor` inside `WaitRegMem` for 7–12 ms every time a frame exceeds 16.67 ms by even 0.5 ms!
- **2. Premature `PendingFenceValue` Expiry When `RecorderFences` Defers EOP Signals to `VkRecorder` (`liverpool.cpp:291, 402-436`):** In `Liverpool::NotePendingFences` (`line 402`) and `PendingFenceValue` (`line 427`), pending EOP/EOS fences are discarded as soon as `rasterizer->DrawPipeReached(it->position)` is true (`// written already: memory holds it`). However, when `Gow3Toggle::RecorderFences` is active (`liverpool.cpp:291`), Stage B (`DrawRec`) does **not** write the EOP fence to guest memory when it reaches `it->position`—it defers `SignalEop(eop)` to Stage C (`VkRecorder` + `Gow3Copy::AfterCopies`) via `SignalAfterHostCopies`! Thus, between `DrawRec` consuming the EOP packet and `VkRecorder` executing it, `PendingFenceValue` falsely returns `false`, forcing `WaitRegMem` (`liverpool.cpp:1329`) to drain `DrawPipe` and spin in `YIELD_GFX()`!
- **3. Stage A/B CPU Saturation & 118,461 Active Inline TSO Checks (`generate_build_ninja.py:217`, `vk_pipeline_cache.cpp:718-1350`, `vk_draw_prep.cpp:47-55`):** At `83e4d4b`, `DrawPreparation` workers are disabled on macOS (`DefaultWorkerCount() == 0` in `vk_draw_prep.cpp:47-55`), so `GpuCommandProcessor` runs `RefreshGraphicsKey` + `StageSpecialization` (`specialization.h:90-185`, allocating 5 `small_vector`s per stage) on every draw, while `CollectRingBindings` (`vk_rasterizer.cpp`) re-copies identical constant buffers per draw. Meanwhile, `generate_build_ninja.py:217` still defaults `GOW3_GUEST_TSO_CHECKS` to `"1"`, keeping 118,461 6-instruction TSO checks active in guest code (+2–3 FPS verified in `LUNA_STATUS_20261025.md` when disabled with `tools/tso_crossing_sites.txt`), and `RecompKernelSema` (`ps4_semaphore.c:383-396`) places `current_count` and `mutex` in the same 128-byte Apple M2 cache line, causing false-sharing wakeups in the new `sema_watch` `ldaxr`/`wfe` loop!

- **4. Cold Shader & Stage-Transition Stalls (`vk_pipeline_cache.cpp:469, 1244`, `texture_cache.cpp:145–198`, `page_manager.cpp:95–190`):** While `83e4d4b` added `SavePipelineCache()` (`vk_pipeline_cache.cpp:469–495`) to persist `vk_pipeline_cache.bin`, cold/transition hitches remain driven by: (a) synchronous `capture_job.Wait()` (`vk_pipeline_cache.cpp:526`) on `GpuCommandProcessor` for every new graphics shader; (b) synchronous compilation on `GpuCommandProcessor` for all **Compute** and **Tessellation** shaders (`compile_async == false` at `vk_pipeline_cache.cpp:1311`); (c) synchronous 15.5 MiB texture pool readbacks (`12.2 ms` with `scheduler.Finish()` when `GOW3_DEFERRED_READBACK` is off); and (d) `mprotect()` page-table lock contention during texture streaming.
- **5. Audio `brbDriver` (`_fn_0x280990`) 82% CPU Burn & `libSceAjm` Premature Speech/Music Cutoffs (`ps4_audioout.c:130–142, 515–538`, `ps4_ajm.c:309–341, 420–484`):** In `ps4_ajm.c`, `at9_init()` (`line 309`) fails to call `ajm_reset_stream_bookkeeping(inst)`, leaving stale `gapless.init.total_samples` with `current.total_samples == 0` on reused instances so `ajm_gapless_end(inst)` immediately terminates decoding, and `at9_decode()` (`line 420`) aborts with `ORBIS_AJM_RESULT_PARTIAL_INPUT` whenever `*prem < inst->superframe_bytes_remain`. Meanwhile, in `ps4_audioout.c`, default `device` pacing returns immediately whenever `ring_fill + length <= ring_limit`, causing `_fn_0x280990` (`code_039.opt.s:15892`) to burst-mix 2–3 grains at 0 ms interval and spin on `RDTSC`.

Figure 1: Empirical Per-Flip Breakdown from docs/handoff/bench/bench_enc.csv & bench_sub.csv at HEAD 83e4d4b



A.1 Reconciliation of All Prior Reports & Recent Commits Up to HEAD 83e4d4b

Subsystem / Claim	State at HEAD 83e4d4b	Verified Source Anchor	Forensic Status & Empirical Measurement
VBlank clock 88.75x slow & relative `nanosleep` drift	**FIXED at** `83e4d4b`	`runtime/modules/libSceVideoOut/ps4_videoout.c:95–145`	`get_time_usec()` now uses `sceKernelGetProcessTime()`, `get_fake_tsc()` uses `recomp_rdtsc()`, and `vblank_thread_func` uses absolute `mach_wait_until(next_abs)`.
Single-threaded H.264 `sceVideodecDecode`	**PARTIALLY FIXED at** `83e4d4b`	`runtime/modules/libSceVideodec/ps4_videodec.c:292–298`	`avctx->thread_count = 4` with `FF_THREAD_SLICE` added. However, single-slice H.264 streams still decode on 1 thread, and double full-frame NV12 `memcpy` remains (`lines 198–266`).
Kernel semaphore broadcast thundering herd & wake latency	**PARTIALLY FIXED at** `83e4d4b` **(has cache-line bug)**	`runtime/kernel/sync/ps4_semaphore.c:414–465, 519–592`	`sceKernelSignalSema` now wakes `signalCount` waiters and `sceKernelWaitSema` uses `sema_watch` (`ldaxr`+`wfe` for 40 us), but `current_count` and `mutex` share a 128B cache line!
Native Vulkan `VkPipelineCache` disk persistence	**IMPLEMENTED at** `83e4d4b` **(needs thread-safety fix)**	`vk_pipeline_cache.cpp:414–495, 800`	`SavePipelineCache()` writes `vk_pipeline_cache.bin` every 60 s and on exit, and loads it on startup. However, `SeedWorkerCaches()` runs after `WarmUp()`, and `WarmUp` races `SavePipelineCache`.

Subsystem / Claim	State at HEAD 83e4d4b	Verified Source Anchor	Forensic Status & Empirical Measurement
`58–75 Vulkan/Metal submissions per frame`	**FALSIFIED at `83e4d4b` (~3 `vkQueueSubmit`/ frame)**	`docs/engineering/SUBMIT_COALESCING_PROPOSAL_20261026.md`, `vk_scheduler.cpp:495`	`GOW3_SUBMIT_TRACE=1` proved there are **~3.0 `vkQueueSubmit` calls/frame** (`Flush`, `PrepareFrame`, `Present`). The 55–60 number in `bench_*.csv` is guest `sceGnmSubmitCommandBuffers` calls!
`200–350 Metal encoders per frame`	**REFINED at `83e4d4b` (~38 encoders/flip in 727-draw scene)**	`docs/engineering/METAL_ENCODERS_20261026.md`, `kosmickrisp-encoder-trace.patch`	`KK_ENCODER_TRACE=1` measured **28.3 render + 9.5 compute = ~38 Metal encoders/flip** (the older 280 number divided by 7.2 FPS xctrace capture rate). 5.1 splits/flip come from tessellation (`prim 14`), 4.4 from copies.
`_fn_0x1a5d10` guest main job spin wait	**Patched (`ldaxr`+`wfe`), still 16% of `guest-main`**	`tools/peephole_guest_code.py:55-86`, `code_025.opt.s:59676`	`wfe` lowers power by ~2 cores, but `guest-main` still waits 16% of gameplay time for `WorkerThread0-4` to finish culling/skinning/PM4 building.
118,461 inline 16-byte TSO crossing checks	**STILL ON BY DEFAULT (`GOW3_GUEST_TSO_CHECKS=1`)**	`tools/generate_build_ninja.py:217`, `tools/tso_crossing_sites.txt`	Keeping the 578 crossing sites (182 functions) in `tools/tso_crossing_sites.txt` and setting `GOW3_GUEST_TSO_CHECKS=0` yields **+2 to +3 FPS** with <10 faults/s, but build generator still defaults to `"1"`.

A.2 Defensible Answers to All 20 Central Questions (Section 2)

#	Central Question	Defensible Source-Level Answer Verified at HEAD 83e4d4b
1	Why does the game plateau around 35–45 FPS in favorable conditions?	Because FIFO/VBlank flip quantization (`driver.cpp:747, 811`) + `port->buffer_labels` locking (`driver.cpp:644-648`) snaps frames finishing in <16.67 ms (~60% of frames in Mount Olympus) to **16.67 ms (60 FPS)** and frames finishing at 17–23 ms (~36% of frames) to **33.33 ms (30 FPS)**, yielding a bimodal mean of **24.2–24.5 ms (40.8–41.3 FPS)**!
2	Is there a hidden frame cap, timing restriction, VSync interaction, or scheduling bottleneck?	**Yes—a severe VBlank/buffer-label quantization interaction!** In `VideoOutDriver::PresentThread` (`driver.cpp:747`), `bool immediate_flips = frame_limit != 0;` is `false` when `frame_limit == 0`. Thus `PresentThread` only pops `requests` and clears `port->buffer_labels[prev_index] = 0` (`line 645`) once every 16.67 ms VBlank tick, stalling `GpuCommandProcessor` in `WaitRegMem` (`liverpool.cpp:1328`) for **6.5–11.8 ms every frame**!
3	Which CPU thread or stage determines the actual critical path?	The critical path is a closed loop: `guest-main` (blocked 14–22 ms in `WaitGpuIdle`, then running ~10 ms of guest logic + 16% job wait in `_fn_0x1a5d10`) -> `GpuCommandProcessor` (Stage A, 85–89% busy, stalled 6.5–11.8 ms/frame in `WaitRegMem`) -> `DrawRec` (Stage B, 59–99% busy) -> `VkRecorder` (61–80% busy) -> `PresentThread` (releasing `buffer_labels[prev_index]`).
4	Is the GPU truly underutilized, or does CPU scheduling leave it waiting?	**CPU scheduling and `WaitRegMem`/`WaitGpuIdle` leave the M2 GPU waiting for 7–12 ms every frame.** While `GpuCommandProcessor` is blocked in `WaitRegMem` waiting for `PresentThread` to clear `buffer_labels[prev_index]` (and while `guest-main` is blocked in `WaitGpuIdle`), no new draws reach the GPU.
5	How much time is consumed by each translation and command-recording layer?	In a 24.5 ms Mount Olympus frame (`bench_enc.csv`): `WaitRegMem` stall on Stage A = **7.02 ms**; Stage A active PM4 + pipeline selection (`GetGraphicsPipeline` + `CollectRingBindings`) = **~10.5 ms**; Stage B (`DrawRec` cache/barriers/ descriptors) = **~11.5 ms** (overlapped with Stage A); `VkRecorder` + KosmicKrisp (`38` Metal encoders, `3` `vkQueueSubmit`s) = **~9.5 ms** (overlapped); `guest-main` active simulation + worker wait = **~9.8 ms** (blocked **14.01 ms** in `submit_lock_ns`).

#	Central Question	Defensible Source-Level Answer Verified at HEAD 83e4d4b
6	Why do some stages drop to 18–25 FPS for several seconds?	When frame work exceeds 16.67 ms on 100% of frames (e.g., 1,500–2,700 draws in heavy combat or during transition streaming), VBlank quantization drops **every** frame to the 33.33 ms (30 FPS) or 50.00 ms (20 FPS) bucket (`bench_sub.csv`: 54.5% at 33.3 ms, 5.6% at 50 ms; `bench_unr.csv`: 29.3 FPS), compounded by synchronous `capture_job.Wait()` shader stalls and 12.2 ms synchronous texture readbacks.
7	What happens when a previously unseen shader is first encountered?	In `PipelineCache::QueueShader` (`vk_pipeline_cache.cpp:498–578`), Stage A copies the GCN binary and user data, dispatches `TranslateProgram` + `StageSpecialization` to `gow3:ShaderCapture0–1`, and **synchronously stalls** `GpuCommandProcessor` **at** `capture_job.Wait()` **(**`line 526`**)**! Worse, for **Compute** and **Tessellation** stages (`line 1311`: `compile_async` is `false`), `CompileModule` runs 100% synchronously on `GpuCommandProcessor`.
8	What happens when a new graphics pipeline must be created?	`GraphicsPipeline::GraphicsPipeline` (`vk_graphics_pipeline.cpp:44–340`) builds a new `VkDescriptorSetLayout` and `VkPipelineLayout` (uncached across pipelines) and invokes `CreateCachedGraphicsPipeline` -> `vkCreateGraphicsPipelines`, where KosmicKrisp translates NIR -> MSL -> `MTLLibrary` -> `MTLRenderPipelineState`.
9	Do pipeline caches actually prevent cold-start stutters?	At `83e4d4b`, `SavePipelineCache()` (`vk_pipeline_cache.cpp:469–495`) now saves `vk_pipeline_cache.bin`, **but two bugs remain**: (1) `SeedWorkerCaches()` (`line 585`) is only called inside `GetGraphicsPipeline` (`line 787`), **never before** `WarmUp()` **/** `StartBackgroundWarmUp()`, and (2) `CreateCachedGraphicsPipeline` (`vk_pipeline_serialization.cpp:256`) still bypasses `pipeline-binaries/` because KosmicKrisp lacks `VK_KHR_pipeline_binary`.
10	Does KosmicKrisp compile shaders or allocate Metal objects synchronously in gameplay?	**Yes.** `vkCreateShaderModule` translates SPIR-V -> NIR synchronously; `vkCreateGraphicsPipelines`/`vkCreateComputePipelines` compiles MSL -> `MTLRenderPipelineState` synchronously on the caller thread; and `vkCreateImage`/`vkCreateSampler` allocate Metal textures/samplers synchronously on `DrawRec`.
11	Are texture streaming and guest-memory aliasing causing repeated GPU/CPU round trips?	**Yes.** Loading Mount Olympus triggers a synchronous **15.5 MiB RGBA32UI texture pool readback taking 12.2 ms** (`scheduler.Finish()` in `TextureCache::ProcessDownloadImages`, `texture_cache.cpp:145–198`), followed by `ApplyBackingWrites` re-uploading 8 overlapped BC1/BC3 textures, plus continuous `mprotect` write-fault traps (`page_manager.cpp:62–190`).
12	Does Vulkan-to-Metal translation create unnecessary barriers, command encoders or submissions?	New instrumentation at `83e4d4b` (`SUBMIT_COALESCING_PROPOSAL_20261026.md` & `METAL_ENCODERS_20261026.md`) proved there are **~3** `vkQueueSubmit`**s/frame** and **~38 Metal encoders/flip** (28.3 render + 9.5 compute) in the 727-draw scene. Of the 9.5 compute encoders, **5.1/flip** are forced by tessellation (`prim 14`, costing ~2.5–3.4 FPS total) and **4.4/flip** by mid-frame `copy_image` / `vkCmdCopyBufferToImage` calls that interrupt render encoders.
13	Is Metal descriptor/heap allocation happening on the frame's critical path?	The `kosmickrisp-cmd-bo-pool.patch` (raising `KK_CMD_POOL_BO_MAX` from `32` to `1024`, enforced by `install_runtime.sh:34–43`) reduced `mtl_new_heap` on `VkRecorder` from **10% to 0%** (`VkRecorder` busy dropped from 84% to 74%). Remaining critical-path descriptor cost is per-draw `UpdateDescriptorSet` on `DrawRec` and `CollectRingBindings` on `GpuCommandProcessor`.
14	Are the game's audio threads spinning, starving, or being scheduled too late?	**Yes (**`brbDriver` **is 82% busy).** In `ps4_audioout.c:130–142, 515–538`, default `device` pacing returns immediately without sleeping whenever `ring_fill + length <= ring_limit` (`2 * len + 1024`), causing `_fn_0x280990` (`code_039.opt.s:15892`) to burst-run 2–3 grains at 0 ms interval and spin polling `RDTSC` (`_fn_0x2e9e90`) in between, while `deadline` pacing (`0312ef0`) caused 651 underruns because its window adjuster steered too slowly after load stalls.

#	Central Question	Defensible Source-Level Answer Verified at HEAD 83e4d4b
15	Why do cutscene audio and visuals drift or hitch?	`83e4d4b` fixed the 88.75x VBlank clock bug and relative `nanosleep` drift in `ps4_videoout.c:95–145`! Remaining cutscene hitching comes from: (1) synchronous H.264 decoding + double 3.1 MB NV12 `memcpy` under `g_decoder_mutex` in `ps4_videodec.c:198–266, 449`; and (2) cutscene frame drops (33.3/50 ms VBlank quantization) delaying video frames relative to CoreAudio's real-time 48 kHz callback.
16	Why can speech or music stop prematurely?	**Two confirmed bugs in** `runtime/modules/libSceAjm/ps4_ajm.c`**:** (1) `at9_init()` (`lines 309–341`) does **not** call `ajm_reset_stream_bookkeeping(inst)`, so a reused `AjmInstance` retains `gapless.init.total_samples != 0` and `current.total_samples == 0`, causing `ajm_gapless_end(inst)` (`line 283`) to immediately abort `at9_decode()` (`lines 416–418`) with 0 samples! (2) `at9_decode()` (`line 420`) aborts with `ORBIS_AJM_RESULT_PARTIAL_INPUT` whenever `*prem < inst->superframe_bytes_remain` across split ring-buffer inputs.
17	Which improvements could raise FPS without sacrificing rendering correctness?	1. Clear `port->buffer_labels[prev_index] = 0` immediately in `SubmitFlipInternal` once `PrepareFrame` has captured the display buffer into `Vulkan::Frame*` (eliminating the 6.5–11.8 ms `WaitRegMem` stall); 2. Fix `PendingFenceValue` (`liverpool.cpp:427`) to track `VkRecorder` fence completion; 3. Enable `GOW3_GUEST_TSO_CHECKS=0` with `tso_crossing_sites.txt` (+2–3 FPS verified); 4. Deduplicate `CollectRingBindings` & memoize `RefreshFlatBuf`/`StageSpecialization`; 5. Fix `RecompKernelSema` 128B cache-line false sharing.
18	Which fixes would improve thermal efficiency on a fanless M2?	1. Fixing `ps4_audioout.c` pacing so `brbDriver` sleeps between 5.33 ms grains instead of burning 82% of a core; 2. Enabling `GOW3_GUEST_TSO_CHECKS=0` (-20% guest instruction count); 3. Padding `RecompKernelSema` so `sema_watch wfe` isn't woken by `mutex` false sharing; 4. Hoisting `copy_image`/`CopyBufferToImage` before render passes to save ~5 TBDR tile store/load cycles per flip.
19	Are there deeper architectural inefficiencies making 60 FPS impractical?	In light-to-medium scenes (700–1,200 draws), eliminating the 7–11 ms `WaitRegMem` (`buffer_labels`) stall and the 14–22 ms `WaitGpuIdle` lockstep brings frame time below 16.67 ms (60 FPS). In heavy combat (2,500–3,100 draws) or hot M2 conditions (~1.5x thermal slowdown), 60 FPS requires combining these CPU/pacing fixes with `SceneTargets` reduced proxy rendering (e.g., 1536x864 or 1280x720 internal targets).
20	Which precise changes should Luna and Codex implement first?	**Codex:** Fix #1 (release `buffer_labels[prev_index]` in `SubmitFlipInternal` after `PrepareFrame` copy), Fix #2 (fix `PendingFenceValue` for `RecorderFences`), Fix #6 (`CollectRingBindings` + `RefreshFlatBuf` deduplication), Fix #8 (hoist texture/alias copies before render passes). **Luna:** Fix #3 (`ps4_ajm.c` gapless/superframe cutoff fix), Fix #4 (`ps4_audioout.c` grain sleep without underruns), Fix #5 (`GOW3_GUEST_TSO_CHECKS=0` default + auto site capture), Fix #7 (`ps4_semaphore.c` 128B cache-line padding).

Part B — Current Architecture Map & Complete Frame Execution Timeline (Investigation I)

Below is the complete 20-stage trace of a single gameplay frame N traveling from `guest-main` through the static ARM64 runtime, `libgow3gpu.dylib` (`shadPS4` Liverpool + Stage A / Stage B / `VkRecorder`), Mesa KosmicKrisp (`09a9fae26d`), Apple Metal, and `VideoOutDriver::PresentThread` at HEAD `83e4d4b`, grounded in the per-flip CSV telemetry (`bench_enc.csv` and `bench_sub.csv`):

Stg	Subsystem Step	Thread Name	Source File & Function (HEAD 83e4d4b)	CPU Dur.	Wait / Stall	Queue / Sync Primitive	Crit. Path?
1	Guest frame start & flip/vblank poll	`guest-main`	`ps4_videoout.c:95–145, 444 shim_sceVideoOutGetFlipStatus`	0.05 ms	0.0 ms	`g_video_table_mutex`, `vh->mutex`	Yes

Stg	Subsystem Step	Thread Name	Source File & Function (HEAD 83e4d4b)	CPU Dur.	Wait / Stall	Queue / Sync Primitive	Crit. Path?
2	Guest simulation, camera & job dispatch	guest-main	code_026.opt.s:30911 _fn_0x1b00c0, ps4_semaphore.c:519	4.2 ms	0.0 ms	sceKernelSignalSema (wakes signalCount)	Yes
3	Visibility culling, skinning & PM4 build	WorkerThread0..4	ps4_threading.c:242-312 (QOS_CLASS_USER_INITIATED)	2.8 ms ea (~45% busy)	0.4–1.2 ms	Writes 0x104xxxxxxx DCBs; sema_watch 40us	Yes
4	Guest main job-completion wait	guest-main	code_025.opt.s:59676-59705 _fn_0x1a5d10 (ldaxr/wfe)	0.1 ms	**1.0–2.8 ms** (16% of main)	Hardware ldaxr/wfe on [x28+x24]	**YES**
5	Guest PM4 DCB/CCB construction	guest-main	ps4_gnmdriver.c:110-580 shim_sceGnmDrawIndex*	1.4 ms	0.0 ms	55–60 GNM command buffers per frame	Yes
6	GNM submit & WaitGpuIdle lockstep gate	guest-main	gow3gpu_gnm_adapter.c:180-238 -> gnmdriver.cpp:74-96, 2322	0.5 ms (memcpy)	**14.0–22.1 ms** (submit_lock_ns)	m_submit_lock, submission_cv (submission_lock)	**YES (GATE)**
7	GPU command processor wakeup	GpuCommandProcessor	liverpool.cpp:150-217 Liverpool::Process	0.1 ms	0.2 ms	submit_mutex, asc_queues[0] (55–60 submits)	Yes
8	Stage A PM4 decode & WaitRegMem **stall**	GpuCommandProcessor	liverpool.cpp:1299-1340 ProcessGraphicsCommands (WaitRegMem)	2.8 ms	**6.5–11.8 ms** (regmem_wait_ns!)	Spins in YIELD_GFX() waiting on buffer_labels / EOP!	**YES (#1 STALL)**
9	Shader/resource metadata & CollectRingBindings	GpuCommandProcessor	vk_pipeline_cache.cpp:785-1350, vk_rasterizer.cpp	**6.8 ms** (DrawPrep=0)	0.0 ms (cold: 5–35 ms)	ConstantRing (32 MB), DrawPipe (16 MB)	**YES (HOT)**
10	Stage B draw packet dequeue & reg delta	gow3:DrawRec	vk_draw_pipe.h:158-216, vk_rasterizer.cpp:2002 RunDrawPacket	1.8 ms	0.3 ms (CpuRelax)	DrawPipe ring (published/consumed)	Yes (tail)
11	Texture/Buffer cache & descriptor bind	gow3:DrawRec	vk_rasterizer.cpp:2500-3180 UpdateDescriptorSet, ScopeResources	**9.5–13.5 ms** (59–99% busy)	0.4 ms (mutex)	TextureCache::mutex, BarrierTracker RB-tree	**YES (HOT)**
12	Deferred DrawCmdBuf chunk replay	gow3:VkRecorder	vk_scheduler.cpp:365-430 RecorderThread	5.5 ms	0.5 ms	recorder_queue (32 KB chunk batching)	Yes (tail)
13	Vulkan SubmitExecution (~3 submits/frame)	gow3:VkRecorder	vk_scheduler.cpp:540-615 SubmitExecution	0.9 ms (3.0x/ frame)	0.5 ms (WaitHostCopies)	Scheduler::submit_mutex, work_semaphore	Yes
14	KosmicKrisp command & unroll translation	gow3:VkRecorder	libvulkan_kosmickrisp.dylib kk_cmd_draw.c	3.8 ms (inline)	0.0 ms	kk_cmd_bo_pool (cap 1024 BOs, 0% heap alloc)	Yes
15	Metal encoder creation (~38 encoders/flip)	gow3:VkRecorder	KosmicKrisp cs_start_render (28.3x) / cs_get_compute (9.5x)	1.6 ms (inline)	0.0 ms	5.1 tessellation splits + 4.4 copy splits/flip	Yes
16	Metal resource & descriptor allocation	DrawRec & VkRecorder	BufferCache, TextureCache, DescriptorHeap -> Metal objects	0.8 ms (cold: 15 ms)	0.0 ms	staging_pool, desc_heap	Yes

Stg	Subsystem Step	Thread Name	Source File & Function (HEAD 83e4d4b)	CPU Dur.	Wait / Stall	Queue / Sync Primitive	Crit. Path?
17	EOP `SubmitFlipInternal` & `PrepareFrame` wait	`GpuCommandProcessor`	`driver.cpp:685–706`, `vk_presenter.cpp:542–562 PrepareFrame`	0.4 ms (FSR+PP)	**1.5–4.0 ms** (`tick_wait_ns`)	Waits for `frames_ahead = 1` (`draw_scheduler.Wait`)	**YES (GATE)**
18	Apple M2 GPU hardware execution	Apple G14G GPU	Hardware tile vertex + fragment + tess compute	11.5–16.5 ms GPU	Starved during `WaitRegMem`!	Fragment-heavy + 38 encoders/flip	Overlapped
19	`PresentThread` 60 Hz wakeup & `buffer_labels` clear	`shadPS4:PresentThread`	`driver.cpp:643–648, 747, 811 Flip`	0.2 ms	**0–16.67 ms VBlank wait!**	Clears `port->buffer_labels[prev_index] = 0`!	**YES (GATE)**
20	Swapchain `Present` to `CAMetalLayer`	`gow3:Present` (`SwapThread`)	`driver.cpp:85–100`, `vk_presenter.cpp:692–880 Present`	0.4 ms	FIFO VSync paced	`swap_mutex`, `present_scheduler` (1 submit/frame)	Async

Figure 2: Multi-Thread Frame Concurrency & VBlank Quantization Critical-Path Diagram at HEAD 83e4d4b (Why Missing

```
Wall Clock (ms): 0.0               6.9              16.67 (VBlank #1)        23.5              33.33 (VBlank #2)
                 |                 |                |                        |                 |
guest-main:      [Sim+Jobs][Wait _fn_0x1a5d10][Submits][Blocked in WaitGpuIdle (submit_lock_ns = 14.0–22.1 ms)... ]
                     \          /
WorkerThread0-4:       [Cull/Skin/PM4 Build (5x USER_INITIATED, 45% busy)]
                              \
GpuCmdProcessor: [PM4 Preamble]   [BLOCKED in WaitRegMem on buffer_labels==0!  ][Stage A: Draws + PrepareFrame]
                                   (regmem_wait_ns = 6.5–11.8 ms!)               \                          \
gow3:DrawRec:    [Drain prior]    [IDLE – Starved by Stage A WaitRegMem!]         [Stage B: Cache+Desc+Barriers ]
                                                                                   \                          \
gow3:VkRecorder: [Submit #1]      [IDLE – Starved by Stage A WaitRegMem!]           [Replay Chunks -> Submit #2  ]
                                                                                    \                          \
Apple M2 GPU:    [Prior Present]  [GPU BUBBLE / STARVED (6.5–11.8 ms)  ]             [38 Encoders: Render+Tess  ]
                                                  ^
PresentThread:   [Sleeping in timer.End() (16.67ms)  ][Wake @ 16.67ms -> Flip(N-1) -> buffer_labels[prev]=0![Sleep..]

ROOT CAUSE OF 40 FPS PLATEAU: Because frame N-1 finished at 17.2 ms (just 0.5 ms after VBlank #0), PresentThread
slept until VBlank #1 (16.67 ms) before calling Flip(N-1) and clearing buffer_labels[prev_index] = 0! That stalls
GpuCommandProcessor in WaitRegMem for 7–11 ms, pushing Frame N past VBlank #1 so it cannot flip until 33.33 ms!
```

Part C & Part D — Frame Critical-Path Reconstruction & 45 FPS Ceiling Investigation (Investigations I & II)

To distinguish an artificial cap from a throughput bottleneck, we analyzed all 8 per-flip CSV traces in `docs/handoff/bench/bench_*.csv` captured at `83e4d4b` alongside `driver.cpp`, `liverpool.cpp`, `gnmdriver.cpp`, and `vk_presenter.cpp`. The table below presents the exact statistical breakdown of every gameplay flip (`draws >= 500`) across all 8 benchmark runs:

Benchmark Run CSV	Flips	Mean ms (FPS)	p50 / p95 / p99 (ms)	Draws / Flip	GNM Subs	`regmem_wait` / Flip	`submit_lock` / Flip	16.7ms / 33.3ms / 50ms %
`bench_diag.csv`	452	**24.19 ms (41.3)**	16.68 / 33.39 / 50.03	706	57.0	**9.87 ms**	**14.47 ms**	**61.9% / 34.3% / 3.1%**
`bench_enc.csv`	422	**24.53 ms (40.8)**	16.68 / 33.40 / 50.07	723	57.2	**7.02 ms**	**14.01 ms**	**59.5% / 36.0% / 3.6%**
`bench_skip.csv`	444	**25.68 ms (38.9)**	16.69 / 50.00 / 50.06	705	57.8	**11.34 ms**	**15.02 ms**	**53.4% / 40.5% / 5.2%**
`bench_ctl2.csv`	486	**27.03 ms (37.0)**	33.30 / 50.00 / 50.04	711	59.8	**9.70 ms**	**15.47 ms**	**47.1% / 45.9% / 6.6%**

Benchmark Run CSV	Flips	Mean ms (FPS)	p50 / p95 / p99 (ms)	Draws / Flip	GNM Subs	regmem_wait / Flip	submit_lock / Flip	16.7ms / 33.3ms / 50ms %
bench_skip2.csv	410	**26.99 ms (37.0)**	33.31 / 33.39 / 50.02	714	59.1	**6.50 ms**	**15.89 ms**	**43.2% / 53.2% / 2.9%**
bench_sub.csv	997	**29.14 ms (34.3)**	33.32 / 50.00 / 83.31	1,503	55.3	**6.67 ms**	**22.06 ms**	**37.5% / 54.5% / 5.6%**
bench_unr.csv	855	**34.17 ms (29.3)**	33.32 / 66.70 / 116.69	1,518	56.0	**11.76 ms**	**20.25 ms**	**39.2% / 37.1% / 14.6%**
bench_ctl.csv	599	**45.16 ms (22.1)**	16.71 / 133.34 / 216.72	1,282	53.5	**19.47 ms**	**28.95 ms**	**50.8% / 14.0% / 10.4%**

- **Proof That 38–45 FPS Arises From Bimodal 16.67 ms / 33.33 ms VBlank Quantization + `buffer_labels` Locking (`driver.cpp:643-648, 747, 811`):** Notice that in `bench_diag.csv` and `bench_enc.csv`, **zero frames actually take 24.5 ms**! Instead, ~60% of frames complete in **16.68 ms (60 FPS)** and ~36% complete in **33.33 ms (30 FPS)**. Why do those 36% jump all the way from ~17.5 ms to 33.33 ms, and why does `regmem_wait_ns` average **7.02 to 11.76 ms per flip**? Because:
  1. In `PatchFlipRequest` (`gnmdriver.cpp:2140-2146`), `sceGnmSubmitAndFlipCommandBuffers` patches a PM4 `WriteData` command that writes `1` into `port->buffer_labels[buf_idx]`.
  2. In `SubmitFlipInternal` (`driver.cpp:685-706`), `presenter->PrepareFrame(group, buffer.address_left)` runs on `GpuCommandProcessor` at the end of Frame N, **blitting/upscaling the guest display buffer (`buffer.address_left`) into an independent host `Vulkan::Frame*` image **and flushing** `draw_scheduler`**. At that exact moment, the GPU has already consumed the guest display buffer for Frame N!**
  3. **However,** `SubmitFlipInternal` **pushes** `Request` **onto** `VideoOutDriver::requests` **and** leaves `port->buffer_labels[prev_index]` set to `1` **until** `PresentThread` **calls** `Flip(request)` **(`driver.cpp:644-648`)!**
  4. **And in** `PresentThread` **(`driver.cpp:747`),** `bool immediate_flips = frame_limit != 0;` **is** `false` **when** `frame_limit == 0`**! Thus** `PresentThread` **sleeps inside** `timer.End()` **(`line 811`) until the next 16.67 ms VBlank tick before calling** `Flip()` **and setting** `port->buffer_labels[prev_index] = 0`**! Meanwhile,** `GpuCommandProcessor` **(decoding Frame N+1) hits** `PM4ItOpcode::WaitRegMem` **(`liverpool.cpp:1299`) polling** `port->buffer_labels[buf_idx] == 0` **and** stalls for 7.0 to 11.8 ms** waiting for `PresentThread`'s 16.67 ms timer!
- **Why Disabling VSync (`Immediate` Mode) Measured Worse (19 FPS) in Earlier Tests:** In `vk_presenter.cpp:692-880`, `SwapThread` (`gow3:Present`) submits a separate command buffer (`present_scheduler.Flush(info)`) on the **same** `vk::Queue` **(`Scheduler::submit_mutex`)** as `VkRecorder`! When `VK_PRESENT_MODE_IMMEDIATE_KHR` was used without pacing, `PresentThread` / `SwapThread` flooded the shared Metal queue with unpaced swapchain blits and `Scheduler::submit_mutex` locks while `CAMetalLayer nextDrawable` contended with the compositor. Keeping `FIFO` on `SwapThread` is right—the bug is holding `port->buffer_labels[prev_index] = 1` until `PresentThread`'s next 16.67 ms wakeup after `PrepareFrame` has already copied the guest buffer!
- **Proof of the `PendingFenceValue` Premature Expiry Bug (`liverpool.cpp:291, 402-436`):** When `DrawRec` (Stage B) processes `PM4ItOpcode::EventWriteEop` (`liverpool.cpp:291`), it defers `SignalEop(eop)` to `VkRecorder` via `rasterizer.SignalAfterHostCopies([eop] { SignalEop(eop); })`. However, `Liverpool::PendingFenceValue` (`liverpool.cpp:427`) stops searching `pending_fences` as soon as `rasterizer->DrawPipeReached(it->position)` is true! Because `DrawRec` advances its `DrawPipe` read pointer (`consumed`) *before* `VkRecorder` executes the chunk and runs `SignalEop`, any `WaitRegMem` polling an EOP fence that is currently in `VkRecorder`'s queue fails `PendingFenceValue`, drains `DrawPipe`, and spins in `YIELD_GFX()`!

Part E — Guest Main Thread, Job Scheduler & Semaphore Analysis (Investigation III)

- `_fn_0x1a5d10` **Spin-Loop Semantics & ABI (**`code_025.opt.s:59676–59705`**,** `tools/peephole_guest_code.py:55–86`**):** In original x86-64 (`0x1a5d10`), the function is a tight polling loop (`cmp dword ptr [rdi], 0 ; jne 0x1a5d10 ; ret`) called from `_fn_0x1b00c0` (`code_026.opt.s:30911`) and 6 other call sites (`code_018/020/022/026/028/031.s`). In the recompiled ARM64 ABI, guest `RDI` is pinned in `x24`, guest `RSP` in `x26`, and `mem_base` in `x28`. At HEAD `83e4d4b`, `tools/peephole_guest_code.py:55–86` replaces `_fn_0x1a5d10` with an `ldaxr w17, [x7] ; cbz w17, L_gow3_wait_done ; wfe ; b L_gow3_wait_load` loop that preserves exact exit flags (`cmp w17, #0`) and pops the guest return address (`ldr x16, [x28, x26] ; add x26, x26, #8 ; ldr x30, [sp], #16 ; ret`).
- **Why** `guest–main` **Still Spends 16% of Gameplay Time in** `_fn_0x1a5d10`**:** As verified in `LUNA_STATUS_20261025.md:143–155`, `_fn_0x1a5d10` is no longer burning a P-core at 100% power thanks to `wfe`, but `guest–main` still waits 16% of the frame for `WorkerThread0–4` to finish their jobs! Why do the 5 workers take so long? Three reasons verified at `83e4d4b`:
  1. **118,461 Inline 16-Byte TSO Crossing Checks Are Still Compiled In by Default (**`tools/generate_build_ninja.py:217`**):** `keep_checks = os.environ.get("GOW3_GUEST_TSO_CHECKS", "1") != "0"` passes `--keep-tso-checks` by default. Even though `tools/tso_crossing_sites.txt` already lists the 578 crossing sites (182 functions) so that building with `GOW3_GUEST_TSO_CHECKS=0` yields **+2 to +3 FPS with <10 faults/s** (`LUNA_STATUS_20261025.md:112–120`), the default build still compiles all 118,461 6-instruction checks!
  2. **128-Byte Cache-Line False Sharing in** `RecompKernelSema` **(**`runtime/kernel/sync/ps4_semaphore.c:383–396, 442–460`**):** At `83e4d4b`, `sceKernelWaitSema` added `sema_watch(&s->current_count, needCount, spin)` (`40 us ldaxr+wfe` watch) to avoid the 30–120 us kernel wakeup latency (~17 times/frame for job workers). However, in `RecompKernelSema` (`lines 383–396`), `int32_t current_count` (offset `40`) and `pthread_mutex_t mutex` (offset `64`) reside in the **exact same 128-byte Apple M2 cache line (Exclusive Reservation Granule)**! Whenever *any* of the 5 workers locks or unlocks `s->mutex` or updates `s->num_waiters` (offset `52`), it invalidates the exclusive reservation on `s->current_count`, prematurely waking every other worker's `wfe` and forcing it to execute a `clock_gettime(CLOCK_MONOTONIC)` syscall (`sema_now_ns()`)!
  3. `WorkerThread0–4` **Contend With 4 GPU Threads on 8 Total Cores (**`ps4_threading.c:180–210`**):** With `guest–main`, `GpuCommandProcessor`, `DrawRec`, and `VkRecorder` at `QOS_CLASS_USER_INTERACTIVE` (occupying the 4 P-cores) and `brbDriver` spinning at 82%, the 5 `WorkerThread0–4` threads (`QOS_CLASS_USER_INITIATED`) Spill onto the 4 slower Blizzard E-cores and experience page-protection write faults (`page_manager.cpp:62`) while writing per-frame buffers.

Part F — Shader Discovery, Cold First-Use Path & Shader Code Quality (Investigations IV, V & VI)

We traced the complete lifecycle of a shader from guest GCN pointer (`regs.ProgramForStage`) through `AmdGpu::GetParams` (`regs_shader.h:269–340`), `PipelineCache::GetProgram` (`vk_pipeline_cache.cpp:1265–1380`), `Shader::TranslateProgram`, `EmitSPIRV`, `CompileSPV` (`vkCreateShaderModule`), and `GraphicsPipeline::GraphicsPipeline` (`vk_graphics_pipeline.cpp:44–340`):

- **Answers to All 17 Shader & Pipeline Cache Questions (Section 8):**
  - **Q1–Q3 (Thread & Blocking):** When `AsyncCompilation()` is enabled (default), unseen graphics shaders still **synchronously block** `GpuCommandProcessor` **(Stage A) inside** `QueueShader` **at** `capture_job.Wait()` **(**`vk_pipeline_cache.cpp:526`**)** while `gow3:ShaderCapture0-1` runs `TranslateProgram` (GCN -> IR), `StageSpecialization`, and `fetch.Capture`! Even worse, for **Compute shaders** and **Tessellation stages** (`vk_pipeline_cache.cpp:1311`: `compile_async = AsyncCompilation() && hw_stage != HwStage::Compute && !sel.regs->stage_enable.hs_en`), `compile_async` is `false`, so `CompileModule` (`TranslateProgram` + `EmitSPIRV` + `CompileSPV`) and `ComputePipeline` creation run **100% synchronously on** `GpuCommandProcessor`! Pipeline creation never runs on `VkRecorder`; it runs on `gow3:ShaderCompile0..3` (for async non-tess graphics) or `GpuCommandProcessor` (for sync/compute/tess).
  - **Q4–Q8 & Q11–Q13 (Caching & Persistence at** `83e4d4b`**):** `Storage::DataBase` (`cache_storage.cpp`) persists `ShaderBinary` (SPIR-V, keyed by `{pgm_hash}_{perm_idx}`, versioned by `ShaderBinaryVersion = 10` in `vk_pipeline_serialization.cpp:34`), `ShaderMeta` (`StageSpecialization` + `Shader::Info`, version `8`), and `PipelineKey` (`g_{hash}` / `c_{hash}`, version `6`). At `83e4d4b`, `PipelineCache::SavePipelineCache()` (`vk_pipeline_cache.cpp:469-495`) also persists the driver's `VkPipelineCache` to `ShaderDir / "vk_pipeline_cache.bin"`. **However**, `CreateCachedGraphicsPipeline` (`vk_pipeline_serialization.cpp:256`) checks `global_key.keySize && EmulatorSettings.IsPipelineCacheEnabled()`; because KosmicKrisp does not expose `VK_KHR_pipeline_binary`, `global_key.keySize == 0`, so `pipeline-binaries/*.bin` is completely unused!
  - **Q9–Q10 (Address Independence & Specialization Multiplication):** Shaders are hashed by `BinaryInfo::shader_hash` (`OrbShdr` footer in `regs_shader.h:260-335`), NOT guest address (`GetParams` uses a thread-local `(VAddr, submission_epoch)` cache only to skip copying 64 KB of guest RAM within the same submission). However, `StageSpecialization` (`specialization.h:76-210`) specializes on every buffer stride/format/swizzle, image view type/sRGB/integer/dst_select, FMask width/height, and sampler `force_degamma`/`force_unnormalized`, creating up to `MaxPermutations = 8` SPIR-V modules per `Program`.
  - **Q14–Q17 (WarmUp Races & Per-Draw Cost):** In `vk_pipeline_cache.cpp:417-436, 585-597`, `SeedWorkerCaches()` (which copies `*pipeline_cache` into `worker_pipeline_caches`) is only called inside `GetGraphicsPipeline()`, **not before** `WarmUp()`, and `SavePipelineCache()` (`line 480`) calls `device.mergePipelineCaches(*merged, {*pipeline_cache, ...})` on a background thread (`gow3:PipelineCacheSave`) **while `WarmUpPool` or `GpuCommandProcessor` is concurrently mutating `*pipeline_cache` in `vkCreateGraphicsPipelines`** (a Vulkan external-synchronization violation!).
- **Per-Draw Redundant Work in Stage A (**`vk_pipeline_cache.cpp:858-1380`**,** `vk_rasterizer.cpp`**):** Because `DrawPreparation` workers are disabled on macOS (`DefaultWorkerCount() == 0` in `vk_draw_prep.cpp:47-55`) after finding a 98% hit rate yielded no net FPS gain when running 4 scanner/prep threads (`LUNA_STATUS_20261025.md:122-128`), `GpuCommandProcessor` runs `GetGraphicsPipeline` on every draw. Inside `GetGraphicsPipeline`, two operations repeat unnecessarily on every draw:
  1. `CollectRingBindings` **(**`vk_rasterizer.cpp` **~4% of Stage A):** Copies small read-only constant buffers (<= 16 KB) into `ConstantRing` (`vk_constant_ring.h`) on every draw without checking if the exact same `(guest_addr, size)` was already copied into `ConstantRing` during the current submission (and has no intervening GPU write)!
  2. `RefreshFlatBuf` **+** `StageSpecialization` **(**`vk_pipeline_cache.cpp:1330-1370`**):** Even when the bound `Program` has only 1 permutation and the 16 user-data SGPRs (`pgm->user_data`) and SRT table generation match the previous draw, `GetProgram` still calls `info.RefreshFlatBuf()` and constructs a full `Shader::StageSpecialization` (allocating 5 `small_vector`s and calling `GetSharp` on every resource)!

- **Shader Code Quality Audit on Apple Silicon (Investigation V):** At `83e4d4b`, `fast_shader_math` (`vk_pipeline_cache.cpp:324–357`) disables `SignedZeroInfNanPreserve` on KosmicKrisp (`ShaderBinaryVersion = 10`), which eliminated the catastrophic 17 ms/frame `precise::pow` slowdown. Remaining shader code quality issues on M2 G14G:
  - `requiredSubgroupSize = 64` **vs. Apple SIMD32 (**`vk_graphics_pipeline.cpp:208–212`**):** GCN shaders expect Wave64 ballotmasks (`u64`). On Apple Silicon (native SIMD-group width 32), KosmicKrisp lowers subgroup ballots/shuffles across SIMD32 lanes.
  - `needs_lds_barriers` **&** `needs_unorm_fixup` **(**`vk_pipeline_cache.cpp:369–372`**):** Enabled for `eMesaKosmickrisp` to preserve workgroup barriers and unorm storage writes.
  - `SamplerSpecialization::force_degamma` **(**`specialization.h:70`**):** Emits scalar `pow(c, 2.2)` per RGB channel in SPIR-V (note: `exp2(log2(x))` rewrite was tested and folded back into `pow` by Mesa NIR; only a polynomial/sqrt approximation `x * x * sqrt(sqrt(x))` changes the emitted MSL, at the cost of a max 0.004 sRGB curve delta).

Part G — Vulkan/Metal Internals, KosmicKrisp & Stage-Transition Hitch Analysis (Investigations VII, VIII, IX & Section 24)

Ground-Truth Update at HEAD `83e4d4b` **(**`METAL_ENCODERS_20261026.md` **&** `SUBMIT_COALESCING_PROPOSAL_20261026.md`**):** Direct in-driver instrumentation at `83e4d4b` resolved two major questions from earlier profiles:

- **1. KosmicKrisp Heap & BO Pool Allocation (Investigation VII):** Applying `scripts/lab/patches/kosmickrisp-cmd-bo-pool.patch` (raising `KK_CMD_POOL_BO_MAX` from `32` to `1024`, now enforced by `scripts/lab/install_runtime.sh:34–43`) reduced `kk_cmd_pool_alloc_bo -> mtl_new_heap -> IOGPUResourceCreate` on `gow3:VkRecorder` from **10% of thread CPU to 0.0%**, cutting `VkRecorder` occupancy from **84% to 74%** and gaining **+0.7 FPS** (`35.6 -> 36.3 FPS` in matched runs).
- **2. Vulkan Submissions & Metal Command Encoders (Investigation VIII):**
  - **Actual** `vkQueueSubmit` **count (**`GOW3_SUBMIT_TRACE=1`**):** Exactly **~3.0** `vkQueueSubmit` **calls per frame** (`1.0` in `Rasterizer::Flush`, `1.0` in `Presenter::PrepareFrame`, `1.0` in `Presenter::Present`, and `0.03` in `Scheduler::Finish`). The `55–60` count in `bench_*.csv` (`submissions`) is guest `sceGnmSubmitCommandBuffers` calls.
  - **Actual Metal Encoder count (**`KK_ENCODER_TRACE=1` **in** `kosmickrisp-encoder-trace.patch`**):** In the 727-draw Mount Olympus scene (`13,543` flips), KosmicKrisp opens **28.3** `MTLRenderCommandEncoder`**s** and **9.5** `MTLComputeCommandEncoder`**s** per flip (**~37.8 Metal encoders/flip total**). The earlier `xctrace` report of 280 encoders/frame was an artifact of dividing by `xctrace`'s 7.2 FPS throttled frame rate.
  - **Why Render Encoders Split (**`cs_get_compute` **call sites):** Every forced split of an open `MTLRenderCommandEncoder` (`5.1 / flip`) is caused by **Tessellation (**`prim 14` **patch list, 5.4 draws/flip)** invoking KosmicKrisp's precompiled compute kernels (`kk_cmd_draw.c:1064, 1979`). Skipping tessellation (`KK_SKIP_TESS=1`) bounds the entire tessellation + split cost at **+2.5 to +3.4 FPS** (`30.0 -> 32.5–33.4 FPS`). The remaining **4.4 compute encoders/flip** come from `copy_image` (`kk_cmd_copy.c:300`, `3.8/flip` from `Runtime::CopyAliasImage`) and `vkCmdCopyBufferToImage` (`kk_cmd_copy.c:94`, `0.6/flip`), which open a compute encoder between render passes.

- **3. Stage-Transition Hitch & Memory Analysis (Investigation IX & Section 24):**
  - **Memory Growth (3.76 GB -> 4.84 GB):** In `LUNA_STATUS_20261025.md:19, 64-67`, `IOAccelerator` memory was measured at `1.74 GB` (menu) -> `3.20 GB` (New Game opening combat, flat thereafter) and `2.03-2.04 GB` (Mount Olympus save, flat over 3 minutes). A code-confirmed leak where `SceneTargets` proxies of deleted `TextureCache` images were never freed was fixed via `SceneTargets::Forget(image_uid)` (`vk_scene_resolution.h:67-70`). Remaining memory growth across multiple stages comes from `TextureCache` GC thresholds (`DEFAULT_PRESSURE_GC_MEMORY = 1.5 GB`, `DEFAULT_CRITICAL_GC_MEMORY = 3.0 GB` in `texture_cache.h:50-53`).
  - **The 15.5 MiB Synchronous Pool Readback Stall (`12.2 ms`):** During stage/save loads, the game renders into a **15.5 MiB** `RGBA32UI` **linear texture pool** on the GPU and then binds 8 `BC1`/`BC3` block-compressed textures overlapping that guest address range (`LUNA_STATUS_20261025.md:53-60`). Because `GOW3_DEFERRED_READBACK` is off by default in `TextureCache` (`texture_cache.h:411-414`: `deferred_readbacks` is only enabled when `GOW3_DEFERRED_READBACK=1`), `ResolveOverlap` / `ProcessDownloadImages` executes a **synchronous 15.5 MiB GPU readback (`scheduler.Finish()`), stalling the entire pipeline for 12.2 ms**, after which `ApplyBackingWrites` marks the 8 BC1/BC3 textures dirty for CPU->GPU re-upload!

Figure 3: Special Requirement (Section 24) — Measured & Source-Verified First-Use Stage-Transition Stutter Timeline at



Part H & Part I — Audio Forensic Analysis & Cutscene Clock Investigation (Investigations X & XI)

We traced the complete audio and cutscene pipeline across `_fn_0x280990` (`code_039.opt.s:15892-16620`), `runtime/modules/libSceAudioOut/ps4_audioout.c`, `runtime/modules/libSceAjm/ps4_ajm.c`, `runtime/modules/libSceVideoOut/ps4_videoout.c`, and `runtime/modules/libSceVideodec/ps4_videodec.c`:

- **1. Complete Audio Flow & Why `brbDriver` (`_fn_0x280990`) Consumes 82% CPU (`code_039.opt.s:15892–16620`, `ps4_audioout.c:130–142, 412–538`):**
  - **Guest Audio Architecture:** `_fn_0x280990` is Scream/Bink's master audio driver loop (`brbDriver`). At `0x2809c4`, it calls `_fn_0x2e9e90` (reading `RDTSC` at 2.13 GHz), converts ticks to double-precision seconds (`VCVTSI2SD`, `VMULSD`, `VDIVSD`, and four 19-instruction `VCVTTSD2SI` blocks at `0x280a0d`, `0x280a23`, `0x280c4d`, `0x280c5f`), mixes voice/BGM channels, submits AT9/MP3 decode batches to `libSceAjm`, and outputs 256-frame (5.333 ms @ 48 kHz) or 512-frame (10.667 ms) PCM grains via `sceAudioOutOutput` / `sceAudioOutOutputs`.
  - **Why `device` Pacing Causes `brbDriver` to Burst-Spin (82% CPU) While `deadline` Pacing (`0312ef0`) Caused 651 Underruns (`ps4_audioout.c:412–538`):**
    - In `device` mode (current default, `audio_deadline_pacing() == 0`), `ps4_audioout_port_output` (`lines 515–538`) calls `audio_port_wait_room`, which **returns immediately without sleeping** whenever `ring_fill + length <= ring_limit` (`ring_limit = 2 * len + 1024`). Because CoreAudio (`audio_render_cb`, `lines 262–280`) pulls **512 frames every 10.667 ms**, each CoreAudio callback drains **two** 256-frame grains at once and broadcasts `g_audio_cond`. `brbDriver` wakes up, pushes Grain 1 (returning in 0 ms), immediately mixes and pushes Grain 2 (returning in 0 ms), and then spins inside `_fn_0x280990` polling `RDTSC` (`_fn_0x2e9e90`) and attempting internal sub-ticks before hitting `audio_port_wait_room` on Grain 3!
    - Conversely, why did `PS4_AUDIO_PACING=deadline` (`audio_port_prepare_deadline`, `lines 412–455`) cause **651 underruns** in `LUNA_STATUS_20261025.md:94`? Look at lines 422–436 of `ps4_audioout.c`: when `ring_fill < 2 * p->length`, `audio_port_prepare_deadline` steers `pace_adjust_ns` by only `-(period_ns / 32)` (`-166 us` per 5.33 ms grain) **and only updates `pace_adjust_ns` once every 32 calls (`AUDIO_PACE_WINDOW_CALLS = 32` = 170 ms)**! Even worse, when `brbDriver` is delayed by a 15 ms CPU/page-fault spike (so `now_ns > deadline_ns`, i.e., the port ring drained close to zero), `audio_port_prepare_deadline` advances `p->next_sync_time_ns = deadline_ns + step_ns`, and on the next grain **still forces `brbDriver` to sleep in `audio_port_sleep_until(deadline)` EVEN WHEN `ring_fill < 2 * p->length` (the ring is starving!)!**
    - The Exact Fix for Both Spin AND Underruns (`ps4_audioout.c:515–538`):** Never sleep in `audio_port_sleep_until` when `ring_fill < 2 * port->length` (let `brbDriver` immediately refill the ring below the 2-grain low-water mark without any deadline sleep!), and only pace `brbDriver` to `period_ns` (5.333 ms via `mach_wait_until`) when `ring_fill >= 2 * port->length`! That guarantees **zero underrun bursts** after a stall while eliminating the 0 ms multi-grain burst-spin when the ring is healthy!
- **2. Confirmed Root Causes of Premature Speech & Music Cutoffs in `libSceAjm` (`runtime/modules/libSceAjm/ps4_ajm.c:309–341, 416–484`):**
  - **Fault 1 — Uninitialized/Stale Gapless State in `at9_init()` (`ps4_ajm.c:309–341`):** When an `AjmInstance` is reused for a new stream and re-initialized via `at9_init(inst, cfg)`, `at9_init()` resets `inst->superframe_bytes_remain` and `inst->at9_num_frames`, **but does NOT reset `inst->gapless` (`ajm_reset_stream_bookkeeping(inst)`)**! When the previous stream finished decoding its `fact.sample_length` samples, `inst->gapless.init.total_samples` was non-zero and `inst->gapless.current.total_samples` reached `0`. On the new stream (if it has no `fact` chunk or before `gapless` sideband is applied), `ajm_gapless_end(inst)` (`line 283`: `init.total_samples != 0 && current.total_samples == 0`) is `true` **on the very first frame**, causing `at9_decode()` (`lines 416–418`) to immediately `break` with 0 samples decoded—permanently cutting off the voice line or music track!
  - **Fault 2 — Over-Strict Superframe Size Check & Desync in `at9_decode()` (`ps4_ajm.c:420–484`):** At line 420, `if (*prem < inst->superframe_bytes_remain) { result |= ORBIS_AJM_RESULT_PARTIAL_INPUT; break; }` rejects any input chunk smaller than `superframe_bytes_remain` (even when `at9_num_frames == 0` and `*prem` contains 1–3 valid AT9 frames, or when a ring buffer wraps across two jobs). Worse, if the game resets a stream's read pointer on a loop/branch without sending `AJM_CONTROL_RESET` while `at9_num_frames != 0`, `superframe_bytes_remain` is out of phase with the new superframe boundary and `Atrac9Decode` fails with `ORBIS_AJM_RESULT_CODEC_ERROR | ORBIS_AJM_RESULT_FATAL` (`line 428`)!

- **3. Cutscene Clock Synchronization Analysis (Investigation XI):**
  - At `83e4d4b`, `ps4_videoout.c:95–145` fixed the 88.75x VBlank clock bug (`get_time_usec` and `get_fake_tsc`) and switched `vblank_thread_func` to absolute `mach_wait_until` deadlines.
  - However, note that **two separate VBlank counters still exist**: `ps4_videoout.c:125` (`vh->vblankStatus.count`, incremented at 60 Hz wall time by `vblank_thread_func`) and `driver.cpp:804` (`main_port.vblank_status.count`, incremented by `shadPS4:PresentThread`). Guest calls to `sceVideoOutGetVblankStatus` and `sceVideoOutAddVblankEvent` route to `ps4_videoout.c`, while `sceVideoOutGetFlipStatus` routes to `ps4_videoout.c:444` (updated via `ps4_videoout_complete_external_flip` when `PresentThread` calls `Flip()`).
  - During cutscenes, audio follows CoreAudio's 48 kHz hardware clock (`audio_render_cb`), while in-engine cinematic rendering steps per completed flip (`sceVideoOutGetFlipStatus`). When cutscene rendering drops from 60 FPS to 30 FPS (due to the 16.67/33.33 ms VBlank quantization in `PresentThread` or H.264 decode copies in `ps4_videodec.c`), flip-driven animations dilate relative to wall-clock audio unless the 7–11 ms `WaitRegMem` (`buffer_labels`) stall is removed.

Part J & Part K — Thermal/Scheduling & Retina/Resolution Analysis (Investigations XII, XIII & XIV)

- **CPU Core Scheduling & Thermal Behavior on the Fanless M2 Air (Investigation XII):**
  - **Core Demand vs. Physical Cores:** The M2 has **4 Avalanche P-cores + 4 Blizzard E-cores**. In gameplay at `83e4d4b`, active CPU demand is ~6.5–7.5 cores: `guest-main` (~85–93% of 1 P-core), `GpuCommandProcessor` (~71–89% of 1 P-core), `DrawRec` (~59–99% of 1 P-core), `VkRecorder` (~61–74% of 1 P-core), `brbDriver` (~82% of 1 core), and `WorkerThread0-4` (~45% x 5 = ~2.25 cores at `USER_INITIATED`).
  - **Why Thermal Throttling Drops FPS by 1.5x (38–45 FPS Cool -> 22–30 FPS Hot):** Without a fan, the M2 MacBook Air's passive chassis can dissipate ~8–10 W continuously. With 4 P-cores + 4 E-cores + GPU active simultaneously, package power starts at ~15–18 W and after 4–6 minutes XNU's thermal manager throttles P-core clocks from `3.49 GHz` down to `~2.2–2.5 GHz` and GPU clocks from `1.4 GHz` down to `~900 MHz`. Because every frame that slows from `16.2 ms` (cool) to `17.5 ms` (warm) misses the 16.67 ms VBlank window in `PresentThread` (`driver.cpp:747, 811`) and snaps to **33.33 ms (30 FPS)**, a **10% thermal clock reduction causes a 50% step-function drop in frame rate (60 FPS -> 30 FPS)**!
- **Retina, Resolution Scaling, FSR & Image Correctness (Investigation XIV):**
  - **Three Distinct Resolutions in the Codebase:**
    1. **Native Guest Internal Resolution (`1920x1080`):** Controlled by the guest engine (with safe-area ratio now fixed to `1.0f` in `ps4_system_service.c` so the game renders full 1920x1080 instead of a centered 1536x864 box).
    2. **Reduced Scene Proxy Resolution (`SceneTargets` in `vk_scene_resolution.h/.cpp`):** Renders eligible 1920x1080 (and 960x540 half-res) scene attachments at a reduced proxy size before upscaling via `TemporalUpscaler` (`vk_temporal_upscaler.cpp`).
    3. **Swapchain / Output Resolution (`vk_presenter.cpp:490–540`, `vk_swapchain.cpp`):** By default, the SDL window is created without `SDL_PROP_WINDOW_CREATE_HIGH_PIXEL_DENSITY_BOOLEAN`, giving a `1710x961` point swapchain scaled by macOS to the `2560x1664` Retina panel. Setting `GOW3_OUTPUT_RES=2560x1440` runs `fsr_pass` + `pp_pass` at 1440p, which measured `35.6–36.3 FPS` vs `38.8–39.8 FPS` at 1080p in Mount Olympus (`–3.2 FPS`).
  - **Root Cause of the 1440p + HiDPI + FSR1 Crash & Visual Artifacts (`vk_presenter.cpp:508–538`, `LUNA_STATUS_20261025.md:27`):** When `GOW3_OUTPUT_RES=2560x1440` is combined with HiDPI / window resize, `frame->width` and `frame->height` in `PrepareFrame` (`vk_presenter.cpp:510–526`) can differ from `swapchain.GetExtent()` when the swapchain recreates in `Present()` (`line 710`), or `GOW3_DIAGNOSTIC_MSAA_SCALING=1` triggers an unaligned MSAA resolve in KosmicKrisp's render encoder.

Part L & Part M — Confirmed Root Causes, Unverified Hypotheses & Mandatory Performance Model (Section 21)

Category & ID	Finding / Hypothesis	Evidence & Falsification Test
Confirmed #1	`VideoOutDriver::PresentThread` (`driver.cpp:644–648, 747, 811`) holds `port->buffer_labels[prev_index] = 1` until the next 16.67 ms VBlank tick (`immediate_flips == false`), stalling `GpuCommandProcessor` in `WaitRegMem` (`liverpool.cpp:1328`) for **6.5–11.8 ms/frame** and quantizing missed frames to **33.33 ms**.	**Evidence:** All 8 `bench_*.csv` files show 1.0 `regmem_wait`/flip (6.5–11.8 ms) and 96%+ of frames quantized to 16.7/33.3/50.0 ms. **Falsification:** Clear `buffer_labels[prev_index] = 0` in `SubmitFlipInternal` right after `PrepareFrame` finishes; `regmem_wait_ns` on `buffer_labels` must drop to ~0 ms.
Confirmed #2	`Liverpool::PendingFenceValue` (`liverpool.cpp:427`) aborts when `DrawPipeReached(it->position)` is true, even though `RunEventWriteEop` (`line 291`) deferred `SignalEop` from `DrawRec` to `VkRecorder` (`SignalAfterHostCopies`).	**Evidence:** Direct code invariant in `liverpool.cpp:291, 402–436`. **Falsification:** Track a monotonic `eop_retired_seq` incremented inside `SignalEop` rather than `DrawPipeReached`; `WaitRegMem` drains on in-flight EOP fences must drop to 0.
Confirmed #3	`RecompKernelSema` (`ps4_semaphore.c:383–396`) places `current_count` (offset 40) and `mutex` (offset 64) in the same 128-byte Apple M2 cache line, causing false-sharing `wfe` wakeups in `sema_watch` (`lines 442–460`).	**Evidence:** `offsetof(RecompKernelSema, current_count) == 40`, `offsetof(RecompKernelSema, mutex) == 64` (both < 128). **Falsification:** Align `mutex` to `alignas(128)` and measure `GOW3_SEMA_STATS=1` spin hits and worker CPU %.
Confirmed #4	`ps4_ajm.c:309–341` (`at9_init`) omits `ajm_reset_stream_bookkeeping(inst)`, leaving `gapless.current.total_samples == 0` on reused instances so `ajm_gapless_end` aborts decoding immediately; `line 420` rejects partial superframes.	**Evidence:** Code path inspection at `ps4_ajm.c:283, 309–341, 416–422`. **Falsification:** Add `ajm_reset_stream_bookkeeping(inst)` to `at9_init()` and log stream completions under `PS4_AJM_LOG=1`.
Confirmed #5	`ps4_audioout.c:412–455, 515–538`: `device` pacing returns in 0 ms when `ring_fill + len <= ring_limit` (causing `brbDriver` to burst-mix and spin at 82% CPU), whereas `deadline` mode sleeps even when `ring_fill < 2 * len` (causing 651 underruns).	**Evidence:** `LUNA_STATUS_20261025.md:94` (651 underruns in `deadline` vs 2 in `device`) + `ps4_audioout.c:422–455`. **Falsification:** Only sleep when `ring_fill >= 2 * len`; verify 0 underruns and `brbDriver` CPU < 25%.
Confirmed #6	`tools/generate_build_ninja.py:217` still defaults `GOW3_GUEST_TSO_CHECKS` to `"1"`, compiling in 118,461 6-instruction TSO checks despite `tools/tso_crossing_sites.txt` already isolating the 578 crossing sites (+2–3 FPS verified in ABBA tests).	**Evidence:** `LUNA_STATUS_20261025.md:112–120` (38.7 -> 41.4 FPS in 834-draw Mount Olympus view). **Falsification:** Build with `GOW3_GUEST_TSO_CHECKS=0` and check `[tso]` fault rate (<10/s).
Hypothesis H1	Unlocking `buffer_labels` after `PrepareFrame` + fixing `PendingFenceValue` will shift ~25–30% of frames in the 710–1,200 draw scenes from the 33.33 ms VBlank bucket into the 16.67 ms bucket (+6 to +10 FPS).	**Test:** Run matched 250 s ABBA benchmark (`bench_cfg.py`) with `GOW3_BENCHMARK_CSV` and compare `regmem_wait_ns`, `submit_lock_ns`, and the `16.7ms %` vs `33.3ms %` flip histogram.

Category & ID	Finding / Hypothesis	Evidence & Falsification Test
Hypothesis H2	Replacing the 12.2 ms synchronous 15.5 MiB `RGBA32UI` pool readback with a GPU-side `vkCmdCopyImageToBuffer` -> `vkCmdCopyBufferToImage` (or enabling `GOW3_DEFERRED_READBACK=1` with `ApplyBackingWrites` ordering) eliminates the load/transition readback stall without BC1/BC3 stripe corruption.	**Test:** Run transition test with `GOW3_TEXTURE_ORDER_DIAG=1` and compare transition frame-time spike and BC texture visual integrity.

Mandatory Critical-Path Performance Model at HEAD 83e4d4b (Section 21)

Pipeline Stage / Metric	Mount Olympus (~720 draws, Cool)	Heavy Combat (~1,500 draws, Cool)	Prolonged Hot M2 (~1,500 draws)	After P0 Fixes (#1–#7)	After P0+P1 Fixes + 80% Proxy
Stage A `WaitRegMem` Stall (`regmem_wait_ns`)	7.02 ms	6.67–11.76 ms	12.5–19.5 ms	**0.4 ms** (Fixes #1, #2)	**0.3 ms**
Stage A Active CPU (`GpuCommandProcessor`)	10.5 ms	14.8 ms	20.5 ms	8.8 ms (Fix #6)	8.2 ms
Stage B Active CPU (`gow3:DrawRec`, overlapped)	11.2 ms (1.8 ms tail)	16.2 ms (2.6 ms tail)	22.8 ms (3.8 ms tail)	10.2 ms (1.4 ms tail)	9.0 ms (1.1 ms tail)
Stage C Active CPU (`gow3:VkRecorder`, overlapped)	9.5 ms (1.2 ms tail)	13.5 ms (1.8 ms tail)	18.5 ms (2.6 ms tail)	8.8 ms (1.0 ms tail)	7.8 ms (0.8 ms tail)
`guest-main` **Active Work + `_fn_0x1a5d10` Wait**	9.8 ms (2.2 ms exposed)	12.4 ms (3.1 ms exposed)	17.5 ms (4.5 ms exposed)	7.8 ms (Fixes #5, #7)	7.5 ms
`guest-main` `WaitGpuIdle` (`submit_lock_ns`)	14.01 ms	20.25–22.06 ms	28.95 ms	**6.5 ms**	**5.2 ms**
`brbDriver` **Audio Thread CPU Occupancy**	82% of 1 core	82% of 1 core	88% of 1 core	**<20% of 1 core** (Fix #4)	**<18% of 1 core**
Apple M2 GPU Execution Time	11.8 ms	15.6 ms	21.8 ms (throttled)	11.2 ms / 14.8 ms	**9.5 ms** (1536x864 proxy)
Unquantized Frame Critical-Path Latency	18.2 ms (straddles 16.7ms!)	23.6 ms	33.8 ms	**12.8 ms / 16.2 ms**	**11.8 ms / 14.5 ms**
VBlank-Quantized Bucket Split (16.7 / 33.3 / 50 ms)	60% / 36% / 4%	38% / 55% / 7%	15% / 55% / 30%	**88% / 12% / 0%** (720d)	**92% / 8% / 0%**
Resulting Mean Frame Rate (FPS)	**40.8–41.3 FPS**	**29.3–34.3 FPS**	**22.1–28.0 FPS**	**52–56 FPS (720d) / 44–48 (1500d)**	**56–60 FPS Sustained**

Why Bottleneck Shifting Matters (Section 21): Notice how in the 720-draw Mount Olympus scene, unquantized critical-path work is **~18.2 ms** (10.5 ms Stage A + 7.0 ms `WaitRegMem` stall + tails). Because 18.2 ms sits right on the 16.67 ms VBlank cliff, **36% of frames miss VBlank #1 and pay a full 33.33 ms**, dragging average FPS down to 40.8 FPS! Removing the 7.0 ms `WaitRegMem` (`buffer_labels` + `PendingFenceValue`) stall drops unquantized frame latency to **~12.8 ms**, shifting nearly all 720-draw frames into the **16.67 ms (60 FPS)** bucket! Conversely, in the 1,500–2,700 draw combat scenes, once `WaitRegMem` is removed, the bottleneck shifts to **Stage B (`DrawRec`, 16.2 ms) and M2 GPU fragment execution (15.6–21.8 ms)**—which is why Stage A/B per-draw reductions (Fix #6) and `SceneTargets` proxy scaling (Fix #16) are required to reach 50–60 FPS in heavy combat.

Part N & Part O — 22 Prioritized Engineering Fixes & Luna vs. Codex Ownership (HEAD 83e4d4b)

ID	Owner	Priority & Target	Exact Source File, Function & Line Range (HEAD 83e4d4b)	Concrete Implementation, Safety Invariants & Cache Rules	Validation Test & Expected Effect
#1	Codex	**P0 (#1 Critical Path):** Unlock `buffer_labels` upon `PrepareFrame` completion instead of waiting for next 16.67 ms VBlank	`third_party/gow3gpu/gpu/shadps4/core/libraries/videoout/driver.cpp:643-648, 685-707, 747, 811` (`Flip`, `SubmitFlipInternal`, `PresentThread`)	In `SubmitFlipInternal` (`driver.cpp:685-706`), `presenter->PrepareFrame(group, buffer.address_left)` has already copied/upscaled the guest display buffer into a private `Vulkan::Frame*` image on `draw_scheduler`. Move the `port->buffer_labels[port->prev_index] = 0; port->SignalVoLabel(); port->prev_index = index;` unlock from `Flip()` (`lines 643-648`) into `SubmitFlipInternal` right after `PrepareFrame` returns (or wake `PresentThread` immediately on `request_cv` by setting `immediate_flips = true` while keeping `SwapThread` FIFO-paced!). **Safety:** `PrepareFrame` records the read of `buffer.address_left` on `draw_scheduler` before subsequent frame draws execute on the same queue, so guest GPU writes to the recycled display buffer are strictly ordered after `PrepareFrame` on the GPU.	Run `bench_cfg.py` and inspect `regmem_wait_ns` and `submit_lock_ns` in CSV. **Eliminates the 6.5–11.8 ms/frame `WaitRegMem` stall on `buffer_labels` (+6 to +10 FPS in 700–1200 draw scenes).**
#2	Codex	**P0 (#2 Critical Path):** Fix `PendingFenceValue` premature expiry when `RecorderFences` is active	`third_party/gow3gpu/gpu/shadps4/video_core/amdgpu/liverpool.cpp:279-301, 401-436, 1299-1340` (`RunEventWriteEop`, `NotePendingFences`, `PendingFenceValue`)	In `Liverpool::pending_fences`, assign each queued fence a monotonic `u64 fence_id` and increment `std::atomic<u64> completed_fence_id` inside `SignalEop` / `RunEventWriteEos` / `RunWriteData` **after** `TryWriteBacking` actually writes guest memory! In `NotePendingFences` (`line 402`) and `PendingFenceValue` (`line 427`), check `it->fence_id <= completed_fence_id.load(std::memory_order_acquire)` instead of `rasterizer->DrawPipeReached(it->position)`. **Safety:** Preserves exact stream ordering while preventing `WaitRegMem` from falsely thinking an EOP fence in `VkRecorder`'s queue was already written.	Verify `RegMem wait` logs under `GOW3_PERF_DIAG=1`: eliminates mid-frame `DrainDrawPipe` + `YIELD_GFX()` stalls on deferred EOP fences.
#3	Luna	**P0 (Audio Cutoff Bug):** Reset gapless state in `at9_init()` and accumulate split superframes in `at9_decode()`	`runtime/modules/libSceAjm/ps4_ajm.c:278-285, 309-341, 416-484` (`ajm_reset_stream_bookkeeping`, `at9_init`, `at9_decode`)	1) Call `ajm_reset_stream_bookkeeping(inst)` inside `at9_init()` (`line 316`) so reused `AjmInstance` slots never retain `gapless.init.total_samples != 0` with `current.total_samples == 0` when re-initialized without a RIFF `fact` chunk. 2) In `at9_decode()` (`line 420`), add a `uint8_t superframe_buf[1024]; uint32_t superframe_fill;` staging buffer to `AjmInstance`: when `*prem < inst->superframe_bytes_remain`, copy `*prem` into `superframe_buf` and consume it cleanly across consecutive `sceAjmBatchJobRun` calls instead of aborting with `ORBIS_AJM_RESULT_PARTIAL_INPUT`.	Run `test_ajm_reinitialize` + cinematic dialogue test with `PS4_AJM_LOG=1`; **eliminates premature speech and music cutoffs**.

ID	Owner	Priority & Target	Exact Source File, Function & Line Range (HEAD 83e4d4b)	Concrete Implementation, Safety Invariants & Cache Rules	Validation Test & Expected Effect
#4	Luna	**P0 (Audio Spin & Underrun Fix):** High-water-only deadline pacing in `ps4_audioout.c`	`runtime/modules/libSceAudioOut/ps4_audioout.c:130-142, 412-470, 515-538` (`audio_port_prepare_deadline`, `ps4_audioout_port_output`)	Why did `device` mode spin `brbDriver` at 82% while `deadline` mode caused 651 underruns? Because `deadline` mode slept even when `ring_fill < 2 * port->length`! Change `ps4_audioout_port_output` (`line 534`) so that: (1) if `port->ring_fill < 2 * port->length` (below 2-grain safety watermark), reset `port->next_sync_time_ns = now_ns + period_ns` and **return immediately without sleeping** so `brbDriver` refills the ring at full speed after any stall; (2) once `port->ring_fill >= 2 * port->length`, sleep until `next_sync_time_ns` using `mach_wait_until()` (exact Mach deadline, zero `nanosleep` slack).	Test with `PS4_AUDIO_LOG=1`: `underruns` remains `<= 2` (same as `device` mode) while `brbDriver` CPU drops from **82% to ~18–25%**.
#5	Luna	**P0 (Guest CPU +2–3 FPS):** Enable `GOW3_GUEST_TSO_CHECKS=0` by default with `tso_crossing_sites.txt` & auto-append tool	`tools/generate_build_ninja.py:217-220`, `tools/peephole_guest_code.py:114-166`, `tools/tso_crossing_sites.txt`	In `tools/generate_build_ninja.py:217`, change `keep_checks = os.environ.get("GOW3_GUEST_TSO_CHECKS", "0") != "0"` so builds use `tools/tso_crossing_sites.txt` (which already protects all 182 functions containing the 578 crossing sites found in gameplay) while dropping inline checks on the remaining 118,461 sites. Add a post-run helper in `scripts/lab/` that merges any new `[tso]` logged sites into `tools/tso_crossing_sites.txt`.	Already verified in ABBA tests (`LUNA_STATUS_20261025.md:117`): **+2.0 to +3.0 FPS** (`38.7 -> 41.4 FPS`) and ~15% lower guest CPU energy.
#6	Codex	**P0 (Stage A Per-Draw CPU):** Memoize `CollectRingBindings` per submission & skip `RefreshFlatBuf`/`StageSpecialization` when unchanged	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_rasterizer.cpp` (`CollectRingBindings`), `vk_pipeline_cache.cpp:1265-1380` (`GetProgram`)	1) In `CollectRingBindings` (~4% of `GpuCommandProcessor`), keep a small 32-entry per-submission table `(guest_addr, size, write_epoch) -> ring_offset`; if a read-only constant buffer (`<= 16 KB`) was already copied into `ConstantRing` in the current submission and `NotePendingGpuWrite` hasn't touched it, reuse `ring_offset` without re-copying! 2) In `Program`, store `last_ud_hash` (CRC/XXH3 of the 16 `user_data` regs + `runtime_info` + `binding` start) when `info.srt_ flattened` has no indirect pointers; if unchanged from the previous lookup, reuse `program->modules[program->last_used]` without calling `RefreshFlatBuf()` or constructing `StageSpecialization`!	Reduces `GpuCommandProcessor` per-draw cost by **~25–35% (-2.5 to -3.5 ms/frame on Stage A)**.
#7	Luna	**P0 (Semaphore False Sharing):** Separate `current_count` and `mutex` onto distinct 128-byte cache lines in `RecompKernelSema`	`runtime/kernel/sync/ps4_semaphore.c:383-396, 442-460` (`RecompKernelSema`, `sema_watch`)	1) Add `__attribute__((aligned(128)))` before `pthread_mutex_t mutex;` in `RecompKernelSema` so `mutex` and `num_waiters` do not share the 128-byte Apple Silicon Exclusive Reservation Granule (ERG) with `current_count`. 2) In `sema_watch` (`line 442`), replace `sema_now_ns()` (`clock_gettime` syscall) inside the `do { ... } while` loop with `mach_absolute_time()` (`cntvct_el0` user-space register read).	Test with `GOW3_SEMA_STATS=1`: increases `sema_watch` catch rate and eliminates spurious `wfe` wakeups across `WorkerThread0-4`.

ID	Owner	Priority & Target	Exact Source File, Function & Line Range (HEAD 83e4d4b)	Concrete Implementation, Safety Invariants & Cache Rules	Validation Test & Expected Effect
#8	Codex	**P1 (Metal Encoders):** Hoist `copy_image` (`CopyAliasImage`) and `vkCmdCopyBufferToImage` before `BeginRendering`	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_rasterizer.cpp:2500-2850` (`ScopeResources`), `docs/engineering/METAL_ENCODERS_20261026.md:23-35`	As proven in `METAL_ENCODERS_20261026.md`, `copy_image` (`3.8/flip`) and `vkCmdCopyBufferToImage` (`1.0/flip`) open a new Metal compute encoder on almost every call because a render encoder was started first. In `Rasterizer::Draw` / `ScopeResources`, issue all pending `SynchronizeAlias` (`CopyAliasImage`) and `RefreshImage` (`CopyBufferToImage`) transfers **before** calling `scheduler.BeginRendering(new_state)` so consecutive copies batch into one compute encoder outside the render pass.	Eliminates ~4–5 render/compute encoder alternations (and TBDR tile store/load pairs) per flip.
#9	Codex	**P1 (Pipeline Cache Thread-Safety & WarmUp):** Synchronize `SavePipelineCache` & seed worker caches before `WarmUp`	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_pipeline_cache.cpp:417-495, 585-597` (`PipelineCache`, `SavePipelineCache`, `SeedWorkerCaches`)	1) Call `SeedWorkerCaches()` at the end of `PipelineCache::PipelineCache()` (`line 444`) right after loading `vk_pipeline_cache.bin` so `WarmUpPool` and `StartBackgroundWarmUp` threads immediately benefit from the loaded driver cache! 2) In `SavePipelineCache()` (`line 480`), do NOT pass `*pipeline_cache` into `device.mergePipelineCaches(*merged, sources)` on `cache_save_thread` while `GpuCommandProcessor` is concurrently mutating `*pipeline_cache` (Vulkan spec requires external synchronization on source caches during `vkMergePipelineCaches` if another thread modifies them); merge on `GpuCommandProcessor` and only do the disk write on `cache_save_thread`.	Prevents driver pipeline-cache race corruption and speeds up `StartBackgroundWarmUp`.
#10	Codex	**P1 (Cold Stutter Fix):** Make Compute & Tessellation shader compilation asynchronous & eliminate `capture_job.Wait()` stall	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_pipeline_cache.cpp:498-540, 820-855, 1311` (`QueueShader`, `GetComputePipeline`, `GetProgram`)	1) At `vk_pipeline_cache.cpp:1311`, allow `compile_async` for `HwStage::Compute` and tessellation stages (snapshot the 64-byte `TessellationDataConstantBuffer` on Stage A so workers don't read live guest memory). 2) In `QueueShader` (`line 526`), copy the bounded SRT guest memory pages (`RefreshFlatBuf`) on `GpuCommandProcessor` (~5 us) so `TranslateProgram` (GCN -> IR) runs asynchronously on `gow3:ShaderCompile*` without blocking `GpuCommandProcessor` at `capture_job.Wait()`!	Eliminates 5–35 ms `GpuCommandProcessor` stalls when new compute, tessellation, or graphics shaders appear during stage transitions.
#11	Codex	**P1 (Transition Hitch Fix):** Replace 15.5 MiB synchronous `RGBA32UI` -> `BC1/BC3` CPU readback with GPU `CopyImageToBuffer`	`third_party/gow3gpu/gpu/shadps4/video_core/texture_cache/texture_cache.cpp:145-198, 750-850` (`ProcessDownloadImages`, `ResolveOverlap`)	As measured in `LUNA_STATUS_20261025.md:17, 53-60`, loading a stage triggers a synchronous 15.5 MiB `RGBA32UI` pool readback (`12.2 ms` with `scheduler.Finish()`) just so 8 overlapped `BC1`/`BC3` textures can re-upload those exact bytes from guest RAM! When a GPU-modified uncompressed image (`RGBA32UI`, 128 bpp) is overlapped by block-compressed textures (`BC1`/`BC3`/`BC7`), copy the source `VkImage` into a device-local transfer `VkBuffer` via `vkCmdCopyImageToBuffer` and populate the overlapped `BC VkImage`s directly via `vkCmdCopyBufferToImage` on the GPU—whilst queueing a deferred background download (`GOW3_DEFERRED_READBACK`) to keep guest RAM coherent without `scheduler.Finish()`!	Eliminates the **12.2 ms synchronous GPU drain** on stage/save loads and prevents stale BC texture stripes.

ID	Owner	Priority & Target	Exact Source File, Function & Line Range (HEAD 83e4d4b)	Concrete Implementation, Safety Invariants & Cache Rules	Validation Test & Expected Effect
#12	Luna	**P1 (VideoDec Copy Elimination):** Decode/scale H.264 NV12 directly into guest framebuffer in `ps4_videodec.c`	`runtime/modules/libSceVideodec/ps4_videodec.c:165-266, 405-505` (`convert_frame_to_nv12`, `write_picture`, `decode_one`)	At `83e4d4b`, `ps4_videodec.c` added `avctx->thread_count = 4` (`FF_THREAD_SLICE`), but `convert_frame_to_nv12` still copies 3.1 MB (1920x1088 NV12) into `instance->output_pixels` and `write_picture` immediately copies that 3.1 MB again into `ctx->mem_base + framebuffer.frame_buffer`, while `decode_one` `malloc`s/`free`s `access_unit`, `AVFrame`, and `AVPacket` on every frame. Write NV12 rows directly into `ctx->mem_base + framebuffer.frame_buffer` (after validating range via `recomp_guest_memory_write`) and keep persistent `AVFrame*`/`AVPacket*` on `Ps4VideodecInstance`.	Eliminates 6.2 MB of redundant memory traffic + 3 heap allocations per cutscene frame.
#13	Luna	**P1 (Guest Codegen):** Replace 19-instruction `VCVTTSD2SI` stack-spill sequence in `peephole_guest_code.py`	`tools/peephole_guest_code.py`, `code_039.opt.s:16046-16075, 16470-16505` (`_fn_0x280990`)	Add a peephole rule in `tools/peephole_guest_code.py` matching the 19-instruction `str q30, [sp, #-16]! ; fcvtzs xDst, dSrc ; fcmeq d30, ... ; ldr q30, [sp], #16` block and replacing it with 4 branchless instructions (`fcvtzs xDst, dSrc ; fcmp dSrc, dSrc ; mov x7, #0x8000000000000000 ; csel xDst, x7, xDst, vs`) without touching `sp` or `q30`.	Removes 15 instructions and 2 stack memory ops per float-to-int conversion in `brbDriver` and physics code.
#14	Codex	**P1 (HiDPI / 1440p Crash Fix):** Fix `PrepareFrame` vs. `Swapchain` extent mismatch & MSAA scaling guard	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_presenter.cpp:505-540, 705-740`, `vk_scene_resolution.cpp`	1) In `Presenter::PrepareFrame`, validate that `frame->width` and `frame->height` match the current target output extent before running `fsr_pass.Render` and `pp_pass.Render`, and recreate the `Frame` image if the window/output resolution changed. 2) Guard `multisample_scaling` in `vk_scene_resolution.cpp` on KosmicKrisp so MSAA attachments use single-sample proxy resolve rather than unsupported MSAA render-encoder scaling.	Fixes the 1440p + HiDPI + FSR1 crash (`LUNA_STATUS_20261025.md:27, 30`) and `scene-resolution-test` MSAA crash.
#15	Codex	**P2 (Pipeline Layout Cache):** Deduplicate `VkDescriptorSetLayout` and `VkPipelineLayout` across pipelines	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_graphics_pipeline.cpp:63-74, 365-408` (`BuildDescSetLayout`)	Cache `VkDescriptorSetLayout` and `VkPipelineLayout` in `PipelineCache` keyed by the hashed `vk::DescriptorSetLayoutBinding` vector instead of creating fresh Vulkan/KosmicKrisp layout objects for every `GraphicsPipeline`.	Reduces pipeline creation overhead and KosmicKrisp argument-table layout allocations.
#16	Codex	**P2 (Thermal / Heavy Combat 60 FPS):** Enable `SceneTargets` 1536x864 (80%) proxy scaling for heavy scenes on fanless M2	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_scene_resolution.h:30-75`, `gow3_settings.cpp:80-150`	In heavy combat (1,500–3,100 draws) or prolonged hot runs on the fanless M2 Air, fragment fill + 16.67 ms VBlank quantization caps native 1080p at 30–36 FPS. Provide an opt-in / adaptive `SceneTargets` preset (`1536x864` or `1600x900`) that scales 3D scene color/depth targets by 80% (36% fewer fragment pixels) while keeping UI and post-processing at full resolution.	Cuts M2 GPU fragment execution time by ~30–35% and reduces SoC thermal dissipation by ~2.5 W.

ID	Owner	Priority & Target	Exact Source File, Function & Line Range (HEAD 83e4d4b)	Concrete Implementation, Safety Invariants & Cache Rules	Validation Test & Expected Effect
#17	Codex	**P2 (Texture Cache Contention):** Thread-local `find_image_cache` on `DrawRec` guarded by `registry_generation`	`third_party/gow3gpu/gpu/shadps4/video_core/texture_cache/texture_cache.h:475–492`, `texture_cache.cpp:850–960` (`FindImage`)	Allow `DrawRec` (Stage B) to probe `find_image_cache` (1024-entry table keyed by `registry_generation.load(std::memory_order_acquire)`) before acquiring `TextureCache::mutex`, taking the lock only on a cache miss or when `IsUpToDate(image_id)` is false.	Removes `TextureCache::mutex` contention between `DrawRec` and guest `SIGBUS` fault handlers.
#18	Codex	**P2 (Page Protection Faults):** Skip re-protecting 1-frame-reuse dynamic ring pages in `0x104xxxxxxx`	`third_party/gow3gpu/gpu/shadps4/video_core/buffer_cache/buffer_cache.cpp:215–245, 290–350` (`NoteHotUse`, `ObtainBuffer`)	`BufferStats::NoteHotUse` (`buffer_cache.cpp:226`) already identifies `0x104xxxxxxx` 64 KB blocks whose reuse distance is 1 frame (written by CPU every frame). Route bindings in those hot 1-frame-reuse blocks to the `Stream` / host-copy path without re-arming `mprotect(PROT_READ)` after every upload.	Eliminates tens of thousands of recurring `mprotect` + `SIGBUS` page faults per second on `WorkerThread0–4`.
#19	Codex	**P2 (Descriptor Redundancy):** Skip redundant `cmdPushDescriptorSetKHR` writes when descriptor payload is unchanged	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_rasterizer.cpp:2850–3160` (`UpdateDescriptorSet`)	Compare the 64-bit hash of the assembled `PushDescriptor` writes against the last pushed descriptor state in the active render pass and skip recording `cmdPushDescriptorSetKHR` when identical.	Reduces `DrawCmdBuf` chunk volume and `VkRecorder` KosmicKrisp descriptor writes by ~20%.
#20	Codex	**P2 (Sampler Cache):** Memoize `TextureCache::GetSampler` on `DrawRec` without `samplers_mutex`	`third_party/gow3gpu/gpu/shadps4/video_core/texture_cache/texture_cache.h:224, 493`, `texture_cache.cpp` (`GetSampler`)	Add a 64-entry direct-mapped `thread_local` sampler cache `(sampler_u64_key, extra_lod_bias_bits) -> vk::Sampler` on `DrawRec` so `GetSampler` avoids locking `samplers_mutex` and probing the robin_map on every sampler binding.	Eliminates per-sampler lock/hash overhead in `UpdateDescriptorSet` (`LUNA_STATUS_20261025.md:79`).
#21	Luna	**P2 (Dual VBlank Cleanup):** Unify guest VBlank equeue delivery with `ps4_videoout.c` absolute clock	`runtime/modules/libSceVideoOut/ps4_videoout.c:104–145`, `third_party/gow3gpu/gpu/shadps4/core/libraries/videoout/driver.cpp:788–810`	Since guest `sceVideoOutAddVblankEvent` and `sceVideoOutGetVblankStatus` are handled by `ps4_videoout.c`, skip the redundant `main_port.vblank_events` loop and `main_port.vblank_cv.notify_all()` in `driver.cpp:788–810` when running under the `gow3gpu` bridge.	Removes duplicate VBlank bookkeeping on `shadPS4:PresentThread`.
#22	Codex	**P3 (Aspect Ratio / HiDPI Polish):** Add optional 16:10 crop/fill & Retina RCAS toggle in `vk_presenter.cpp`	`third_party/gow3gpu/gpu/shadps4/video_core/renderer_vulkan/vk_presenter.cpp:750–840` (`Present`)	Provide `GOW3_ASPECT_FILL=1` in `SetExpectedGameSize` to fill the 16:10 (`2560x1664`) MacBook Air panel without top/bottom letterbox bars (`CHATGPT_HANDOFF_20261025.md:149–156`) .	Completes Goal 2 edge-to-edge presentation option with zero CPU cost.

Part P — Five Immediate Next Experiments to Run at HEAD 83e4d4b

Exp #	Target Hypothesis & Command / 5-Line Patch	What to Measure (`GOW3_BENCHMARK_CSV` / Logs)	Success Criterion
Exp 1	**Test Finding #1 (`buffer_labels` VBlank Stall):** Run `docs/handoff/bench/windows.sh fps60 GOW3_FPS_LIMIT=60` (which sets `frame_limit = 60` -> `immediate_flips = true` in `driver.cpp:747`!) or move `port->buffer_labels[prev_index] = 0` into `SubmitFlipInternal`.	Compare `regmem_wait_ns`, `submit_lock_ns`, and `16.7ms %` vs `33.3ms %` in `bench_fps60.csv` against `bench_enc.csv`.	`regmem_wait_ns` drops from **7.0–11.8 ms to <1.5 ms**; 16.7 ms bucket rises from ~60% to >85%; mean FPS rises by **+5 to +9 FPS**.

Exp #	Target Hypothesis & Command / 5-Line Patch	What to Measure (`GOW3_BENCHMARK_CSV` / Logs)	Success Criterion
Exp 2	**Test Finding #2 (`PendingFenceValue` EOP Stall):** Run `docs/handoff/bench/windows.sh norecfence GOW3_DISABLE_OPTIMIZATIONS=1024` (disables `RecorderFences`, keeping `SignalEop` on `DrawRec` so `DrawPipeReached` matches memory writes) or apply Fix #2 (`completed_fence_id`).	Compare `regmem_wait_ns` and `RegMem wait` diagnostic lines (`GOW3_PERF_DIAG=1`).	Confirms how many milliseconds of `regmem_wait_ns` come from `RecorderFences` premature `PendingFenceValue` expiry vs `buffer_labels`.
Exp 3	**Test Finding #5 (`GOW3_GUEST_TSO_CHECKS=0`):** Run `GOW3_GUEST_TSO_CHECKS=0 python3 tools/generate_build_ninja.py && ninja ps4_app` and run an interleaved ABBA test on the Mount Olympus save.	Measure mean FPS, `[tso]` crossing fault rate, and `WorkerThread0-4` / `guest-main` CPU %.	Re-confirms **+2 to +3 FPS** with `<10 faults/s` (`tools/tso_crossing_sites.txt`) and lower thermal throttling.
Exp 4	**Test Finding #3 & #4 (Audio `ps4_ajm.c` & `ps4_audioout.c`):** Apply Fix #3 (`ajm_reset_stream_bookkeeping` in `at9_init`) and Fix #4 (only sleep when `ring_fill >= 2 * len`), then run with `PS4_AUDIO_LOG=1 PS4_AJM_LOG=1`.	Check `underruns`, `repeated_grains`, `brbDriver` CPU %, and cutscene dialogue continuity.	`underruns <= 2`, `brbDriver` CPU drops from **82% to <25%**, and zero premature voice/music cutoffs.
Exp 5	**Test Finding #11 (Stage-Transition 15.5 MiB Readback):** Run a save-load / stage-transition capture with `GOW3_DEFERRED_READBACK=1 GOW3_TEXTURE_ORDER_DIAG=1`.	Measure load-time `scheduler.Finish()` stall (`t_readback_wait`) and check `order_stale_binds`.	The **12.2 ms** synchronous readback stall disappears; verify whether `ApplyBackingWrites` needs GPU-side `CopyImageToBuffer` (Fix #11) to avoid stale binds.